

LAPORAN PRAKTIKUM

STRATEGI ALGORITMA

PRAKTIKUM 6: ALGORITMA BFS DAN DFS

I. DEFINISI

1.1 Traversal Graf

Traversal graf adalah proses mengunjungi setiap simpul (node) dalam sebuah graf secara sistematis. Terdapat dua pendekatan representasi graf dalam traversal: graf statis (sudah terbentuk sebelum pencarian, direpresentasikan sebagai struktur data) dan graf dinamis (terbentuk selama proses pencarian berlangsung). Algoritma traversal dikategorikan sebagai uninformed/blind search karena tidak menggunakan informasi heuristik untuk memandu pencarian.

1.2 Breadth First Search (BFS)

Definisi: BFS adalah algoritma traversal graf yang mengunjungi semua simpul secara berlapis (level by level), dimulai dari simpul awal v , kemudian seluruh tetangganya, lalu tetangga dari tetangganya, dan seterusnya.

Cara Kerja: Menggunakan struktur data antrian (queue). Simpul awal dimasukkan ke antrian, lalu setiap simpul yang dikeluarkan dari depan antrian akan memasukkan semua tetangganya yang belum dikunjungi ke belakang antrian.

Fungsi: Menemukan jalur terpendek pada graf tidak berbobot, eksplorasi jaringan sosial, web crawling, dan level-order traversal pada pohon.

Kelebihan: Menjamin jalur terpendek pada graf tidak berbobot, mengeksplorasi graf secara menyeluruh per level, dan tidak berisiko stack overflow.

Kekurangan: Membutuhkan memori $O(V)$ untuk antrian, tidak efisien untuk graf sangat lebar karena antrian bisa sangat besar, dan tidak efisien untuk mencari jalur dalam pada graf besar.

1.3 Depth First Search (DFS)

Definisi: DFS adalah algoritma traversal graf yang mengeksplorasi setiap cabang sedalam mungkin sebelum backtrack ke simpul sebelumnya dan mencoba cabang lain, dimulai dari simpul awal v .

Cara Kerja: Menggunakan rekursi (atau stack). Dari simpul yang dikunjungi, langsung masuk ke tetangga pertama yang belum dikunjungi, terus ke dalam hingga tidak ada tetangga baru, baru kemudian backtrack.

Fungsi: Deteksi siklus dalam graf, topological sorting, penyelesaian puzzle dan labirin, pencarian komponen terhubung, dan path finding pada graf berarah.

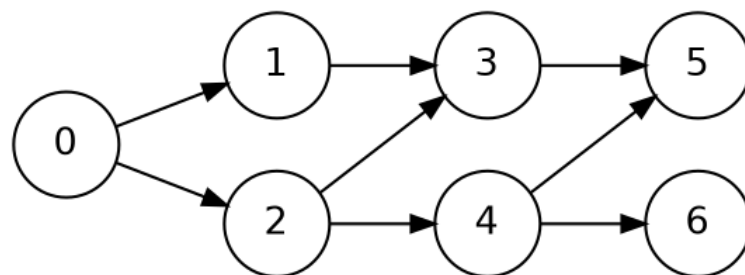
Kelebihan: Hemat memori $O(H)$ di mana H adalah kedalaman rekursi, efisien untuk eksplorasi jalur dalam, dan mudah diimplementasikan secara rekursif.

Kekurangan: Tidak menjamin jalur terpendek, berisiko stack overflow pada graf sangat dalam, dan dapat terjebak dalam siklus jika tidak ada penandaan simpul yang dikunjungi.

II. PRE TEST

Pertanyaan 1: Analisis graf berarah pada Gambar 6.4 dan tuliskan urutan jalurnya secara manual apabila menggunakan algoritma BFS dan DFS mulai dari simpul/node ke-0!

Graf Gambar 6.4 (7 simpul): Edge: $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 5$, $4 \rightarrow 5$, $4 \rightarrow 6$



Graf Gambar 6.4 (Pre Test & Post Test)

Gambar 2.1 Visualisasi Graf Pre Test (Gambar 6.4)

Jawaban BFS manual dari node 0:

Langkah	Node Diproses	Antrian Setelah	Alasan
1	0	[1, 2]	kunjungi 0, masukkan tetangga 1 dan 2

2	1	[2, 3]	kunjungi 1, masukkan tetangga 3
3	2	[3, 4]	kunjungi 2, masukkan 4 (3 sudah di antrian)
4	3	[4, 5]	kunjungi 3, masukkan tetangga 5
5	4	[5, 6]	kunjungi 4, masukkan 6 (5 sudah di antrian)
6	5	[6]	kunjungi 5, tidak ada tetangga baru
7	6	[]	kunjungi 6, tidak ada tetangga baru

Urutan BFS: 0, 1, 2, 3, 4, 5, 6

Jawaban DFS manual dari node 0:

Langkah	Node	Aksi
1	0	kunjungi 0, rekursi ke tetangga pertama: 1
2	1	kunjungi 1, rekursi ke tetangga: 3
3	3	kunjungi 3, rekursi ke tetangga: 5
4	5	kunjungi 5, tidak ada tetangga, backtrack ke 3, ke 1, ke 0
5	2	dari 0 rekursi ke tetangga berikutnya: 2
6	4	kunjungi 2, rekursi ke 4 (3 dan 5 sudah dikunjungi)
7	6	kunjungi 4, rekursi ke 6 (5 sudah dikunjungi)

Urutan DFS: 0, 1, 3, 5, 2, 4, 6

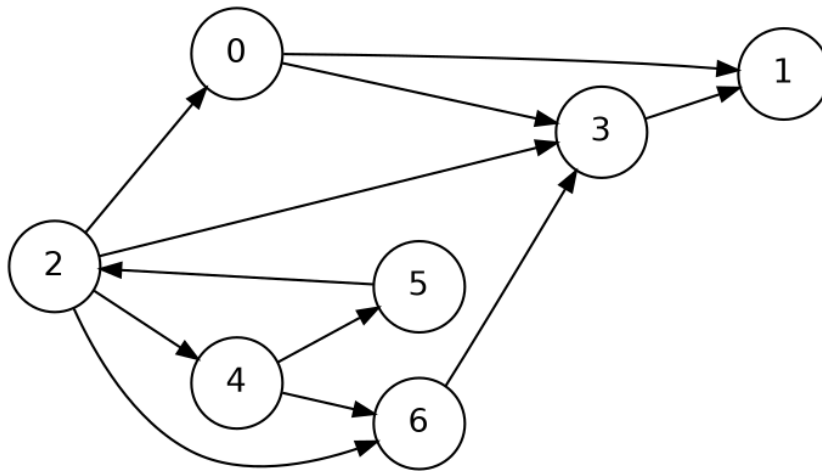
Pertanyaan 2: Apa perbedaan utama antara algoritma BFS dan DFS?

Aspek	BFS	DFS
Strategi penelusuran	Melebar (level by level)	Mendalam (cabang habis dulu)
Struktur data	Queue (antrian)	Stack / Rekursi
Kompleksitas memori	$O(V)$ untuk antrian	$O(H)$, H = kedalaman rekursi
Jalur terpendek	Menjamin pada graf tidak berbobot	Tidak menjamin
Penggunaan utama	Shortest path, web crawling	Deteksi siklus, topological sort
Kompleksitas waktu	$O(V + E)$	$O(V + E)$

III. ANALISIS PRAKTIKUM

3.1 Langkah Praktikum: BFS dan DFS pada Graf 6.1

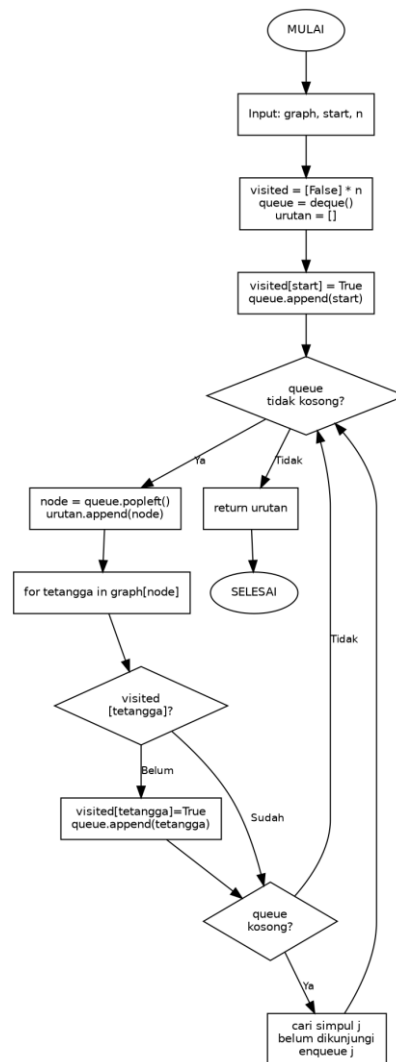
Visualisasi Graf 6.1



Graf Gambar 6.1 (Langkah Praktikum)

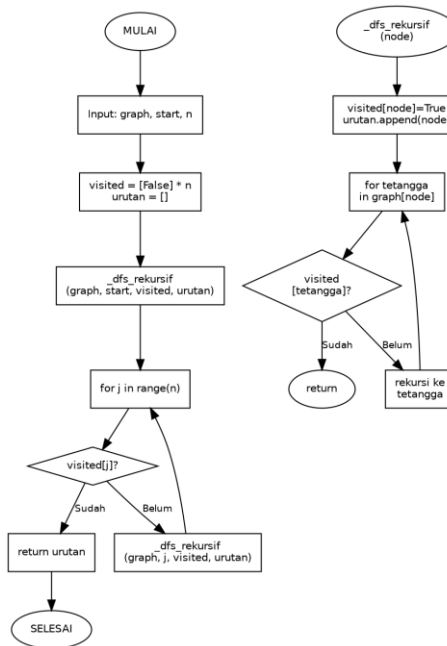
Gambar 3.1 Visualisasi Graf Berarah Gambar 6.1 (Langkah Praktikum)

Flowchart BFS



Gambar 3.1 Flowchart Algoritma BFS

Flowchart DFS



Gambar 3.2 Flowchart Algoritma DFS

Source Code Python: BFS

```

def bfs(graph, start, n):
    visited = [False] * n
    urutan = []
    queue = deque()

    # tandai simpul awal sebagai dikunjungi dan masukkan ke antrian
    visited[start] = True
    queue.append(start)

    while queue:
        # ambil simpul paling depan antrian
        node = queue.popleft()
        urutan.append(node)

        # kunjungi semua tetangga yang belum dikunjungi
        for tetangga in graph[node]:
            if not visited[tetangga]:
                visited[tetangga] = True
                queue.append(tetangga)

        # jika antrian kosong tapi masih ada simpul yang belum dikunjungi
        if not queue:
            for j in range(n):
                if not visited[j]:
                    visited[j] = True
                    queue.append(j)
                    break

    return urutan

if __name__ == "__main__":
    # graf berarah Gambar 6.1 sesuai kode tester modul
    n = 7
    graph = {
        0: [1, 3],
        1: [],
        2: [0, 3, 4, 6],
        3: [1],
        4: [5, 6],
        5: [2],
        6: [3],
    }
  
```

```

    }

    hasil = bfs(graph, 0, n)
    print("BFS mulai dari node 0:", hasil)

```

Source Code Python: DFS

```

def _dfs_rekursif(graph, node, visited, urutan):
    # tandai simpul saat ini sebagai dikunjungi
    visited[node] = True
    urutan.append(node)

    # kunjungi semua tetangga yang belum dikunjungi secara rekursif
    for tetangga in graph[node]:
        if not visited[tetangga]:
            _dfs_rekursif(graph, tetangga, visited, urutan)

def dfs(graph, start, n):
    visited = [False] * n
    urutan = []

    # mulai DFS dari simpul awal
    _dfs_rekursif(graph, start, visited, urutan)

    # tangani simpul yang belum terjangkau
    for j in range(n):
        if not visited[j]:
            _dfs_rekursif(graph, j, visited, urutan)

    return urutan

if __name__ == "__main__":
    # graf berarah Gambar 6.1 sesuai kode tester modul
    n = 7
    graph = {
        0: [1, 3],
        1: [],
        2: [0, 3, 4, 6],
        3: [1],
        4: [5, 6],
        5: [2],
        6: [3],
    }

    hasil = dfs(graph, 0, n)
    print("DFS mulai dari node 0:", hasil)

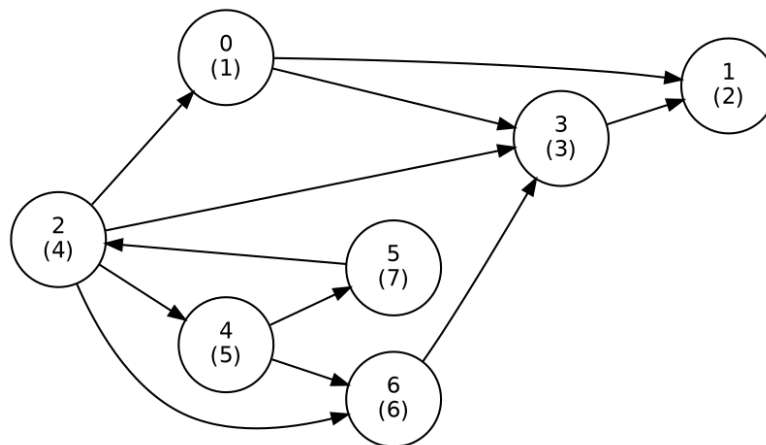
```

Analisis Output dan Perbedaan Kedua Algoritma

Program dijalankan pada graf berarah Gambar 6.1 (7 simpul) dimulai dari node 0. Graf memiliki edge: $0 \rightarrow 1$, $0 \rightarrow 3$, $2 \rightarrow 0$, $2 \rightarrow 3$, $2 \rightarrow 4$, $2 \rightarrow 6$, $3 \rightarrow 1$, $4 \rightarrow 5$, $4 \rightarrow 6$, $5 \rightarrow 2$, $6 \rightarrow 3$.

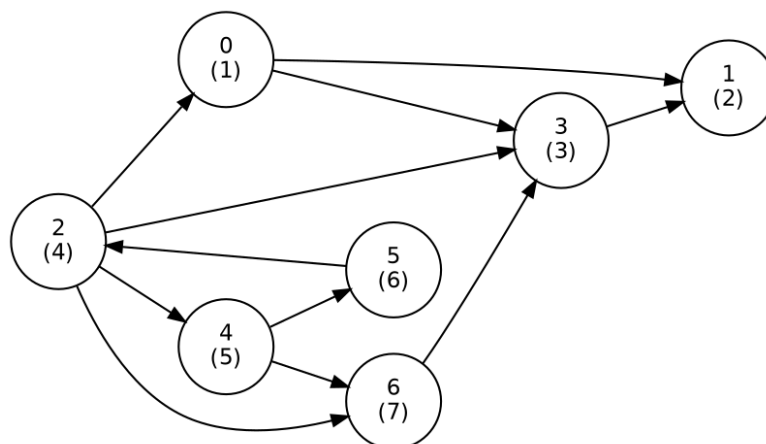
Algoritma	Urutan Traversal	Simpul ke-5 dikunjungi ke-	Simpul ke-6 dikunjungi ke-
BFS	[0, 1, 3, 2, 4, 6, 5]	ke-7 (terakhir)	ke-6
DFS	[0, 1, 3, 2, 4, 5, 6]	ke-6	ke-7 (terakhir)

Visualisasi Urutan Traversal



Urutan BFS dari Node 0 (Graf 6.1)

Gambar 3.2 Urutan BFS dari Node 0 (angka dalam kurung = urutan kunjungan)



Urutan DFS dari Node 0 (Graf 6.1)

Gambar 3.3 Urutan DFS dari Node 0 (angka dalam kurung = urutan kunjungan)

Perbedaan terlihat pada urutan kunjungan simpul 5 dan 6. BFS mengunjungi simpul 6 sebelum 5 karena 6 masuk antrian lebih dulu (saat memproses simpul 4, edge 4→6 diperiksa sebelum 4→5 jika indeks lebih kecil diutamakan). DFS sebaliknya, masuk ke simpul 5 terlebih dahulu mengikuti cabang 4→5 sebelum backtrack ke 4→6.

3.2 Post Test: BFS dan DFS pada Graf Pre Test (Gambar 6.4)

Source Code Python

```

from collections import deque

def bfs(graph, start, n):
    visited = [False] * n
    urutan = []
    queue = deque()

    # tandai simpul awal dan masukkan ke antrian
    visited[start] = True
    queue.append(start)
  
```

```

while queue:
    node = queue.popleft()
    urutan.append(node)

    # kunjungi tetangga yang belum dikunjungi
    for tetangga in graph[node]:
        if not visited[tetangga]:
            visited[tetangga] = True
            queue.append(tetangga)

    # tangani simpul tidak terjangkau
    if not queue:
        for j in range(n):
            if not visited[j]:
                visited[j] = True
                queue.append(j)
                break

return urutan

def _dfs_rekursif(graph, node, visited, urutan):
    # tandai dan catat simpul yang dikunjungi
    visited[node] = True
    urutan.append(node)

    # rekursif ke tetangga yang belum dikunjungi
    for tetangga in graph[node]:
        if not visited[tetangga]:
            _dfs_rekursif(graph, tetangga, visited, urutan)

def dfs(graph, start, n):
    visited = [False] * n
    urutan = []

    _dfs_rekursif(graph, start, visited, urutan)

    # tangani simpul yang belum terjangkau
    for j in range(n):
        if not visited[j]:
            _dfs_rekursif(graph, j, visited, urutan)

    return urutan

```

Jawaban 1: Penerapan BFS dan DFS pada graf Gambar 6.4

Program dijalankan pada graf pre test (Gambar 6.4) dengan 7 simpul mulai dari node 0:

Algoritma	Hasil Program	Jawaban Manual Pre Test	Sama?
BFS	[0, 1, 2, 3, 4, 5, 6]	[0, 1, 2, 3, 4, 5, 6]	Ya
DFS	[0, 1, 3, 5, 2, 4, 6]	[0, 1, 3, 5, 2, 4, 6]	Ya

Jawaban 2: Analisis kesesuaian hasil program dengan jawaban pre test

Hasil program sepenuhnya sesuai dengan jawaban manual pre test untuk kedua algoritma.

Ini memvalidasi bahwa:

1. Implementasi Python BFS menggunakan deque sudah benar dalam mengeksplorasi graf secara berlapis, menghasilkan urutan yang identik dengan penelusuran manual menggunakan antrian.
2. Implementasi Python DFS rekursif sudah benar dalam mengeksplorasi graf secara mendalam, menghasilkan urutan yang identik dengan penelusuran manual menggunakan prinsip backtracking.
3. Penanganan simpul tidak terjangkau (isolated nodes) sudah ditangani dengan benar di kedua implementasi sehingga seluruh 7 simpul berhasil dikunjungi.

IV. IMPLEMENTASI NYATA

Algoritma BFS dan DFS merupakan fondasi dari banyak aplikasi dunia nyata mulai dari navigasi, jaringan sosial, hingga analisis jaringan komputer. Jurnal berikut mendukung relevansinya.

[1] **Penulis:** Ahmed, R., Chekol, B., dan Alsaleh, M. (2025)

Judul: *"HDBMS: A Context-Aware Hybrid Graph Traversal Algorithm for Efficient Information Discovery in Social Networks"*

Publikasi: arXiv preprint arXiv:2508.14092, Uskudar University, Istanbul, 2025.

Intisari: Makalah ini mengusulkan algoritma traversal hibrid berbasis BFS dan DFS yang dinamis dan adaptif untuk jaringan sosial. Eksperimen pada berbagai graf berarah menunjukkan bahwa BFS unggul dalam menemukan jalur terpendek antar pengguna (misal pencarian teman terdekat), sedangkan DFS lebih efisien dalam menelusuri rantai hubungan yang dalam (misal rantai influencer). Karya ini menegaskan bahwa pemilihan algoritma traversal harus disesuaikan dengan konteks dan struktur graf yang dihadapi.

Link: <https://arxiv.org/pdf/2508.14092>

Kelebihan dalam konteks nyata:

- **BFS** unggul untuk pencarian jalur terpendek pada jaringan tidak berbobot seperti jaringan sosial (degree of separation), routing paket jaringan, dan rekomendasi koneksi.

- **DFS** unggul untuk eksplorasi mendalam seperti deteksi siklus dalam dependency graph, topological sort pada build system, dan traversal struktur file sistem operasi.
- Kompleksitas $O(V+E)$ yang sama membuat keduanya efisien untuk graf skala menengah dan dapat dioptimalkan lebih lanjut dengan teknik paralel untuk graf berskala besar.

Kekurangan dalam konteks nyata:

- BFS membutuhkan memori besar $O(V)$ untuk antrian, tidak cocok untuk graf dengan jutaan simpul seperti graf internet tanpa optimasi memori.
- DFS berisiko stack overflow pada graf sangat dalam apabila diimplementasikan secara rekursif, dan perlu diubah ke iteratif berbasis stack eksplisit untuk sistem produksi.